Voice Support Copilot

Case Study — Technical Overview & Design Rationale

1. Architecture

Modular engine-based design orchestrated by a central SupportCopilot controller:

- LLM Engine — Qwen/Qwen3-1.7B via Transformers for efficient local inference
- Voice Engine — Hybrid STT/TTS pipeline
  - STT: Vosk (local, offline) for low latency and data privacy
  - TTS: edge-tts (cloud) for high-quality neural voice synthesis
- RAG Engine — ChromaDB + all-MiniLM-L6-v2 embeddings for semantic document retrieval

2. Assumptions & Tradeoffs

Environment: Low-latency settings where response speed drives user engagement
Hardware: Standard CPU workstation; GPU auto-detected when available
Key tradeoffs:

- 1.7B model over 7B+ — near-instant responses on standard hardware; slight reasoning tradeoff
- Vosk over Whisper — minimal CPU overhead for live streams vs. higher accuracy at higher cost

3. Future Improvements

- Agentic loop (ReAct) for multi-step actions like order lookups
- Fine-tuning on support transcripts for domain-specific tone
- Streaming TTS to reduce perceived latency further

4. Code Structure

main.py: Central orchestration and CLI loop
engine_arc/llm_engine.py: LLM pipeline management
engine_arc/rag_engine.py: Document ingestion and vector search
engine_arc/voice_engine.py: STT and TTS logic
prompt/prompt.py: System instructions and ticket structuring

5. How It Works

- Startup: Documents in knowledgebase/ are parsed into the vector store
- Interaction: User types a query or presses 'v' to speak
- Response: RAG retrieves context; LLM generates a grounded reply
- Exit: Full session is summarized into a structured support ticket

6. Evaluation Criteria

- AI Engineering — RAG, local LLM, and voice pipeline integration
- Prompt Design — Distinct system and summary prompts for persona and output control
- Product Thinking — Complete support lifecycle from query to structured ticket
- Practicality — Local-first approach for cost-effective, low-dependency deployment